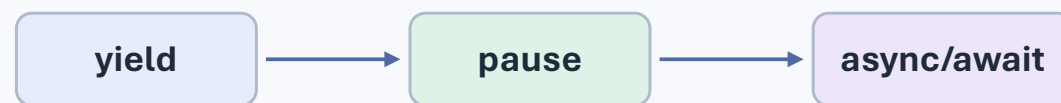


Использование генераторов для асинхронного программирования в Python

Краткий путь от обычного `yield` к корутинам, `async/await` и асинхронным генераторам



одна идея: приостановить → продолжить

Python • generators • async

Зачем нужна асинхронность

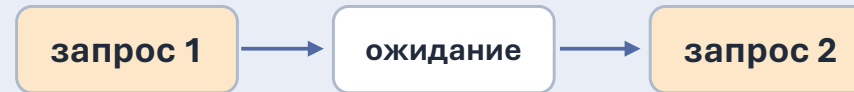
Программа часто не считает, а ждёт: сеть, файлы, базу данных, таймеры.

Синхронный код блокируется во время ожидания.

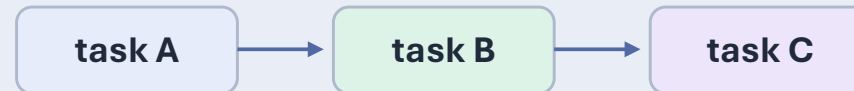
Асинхронный код может переключаться между задачами.

Идея: не простаивать во время ожидания.

Синхронно: задача стоит в очереди



Асинхронно: ожидание освобождает место



пока A ждёт — выполняются B и C

Что такое генератор

Генератор — функция с `yield`.
Он выдаёт значения постепенно, а не сразу.
После `yield` сохраняется строка выполнения и локальные переменные.

```
def numbers():  
    yield 1  
    yield 2  
    yield 3  
  
for n in numbers():  
    print(n)
```

Состояние генератора



`next()` каждый раз продолжает выполнение

Почему `yield` похож на асинхронную паузу

`yield` временно останавливает функцию.
Внешний код решает, когда продолжить генератор.
Это похоже на задачу, которая добровольно отдаёт управление.



Генераторы как корутины

Корутина — функция, которую можно приостанавливать и возобновлять.

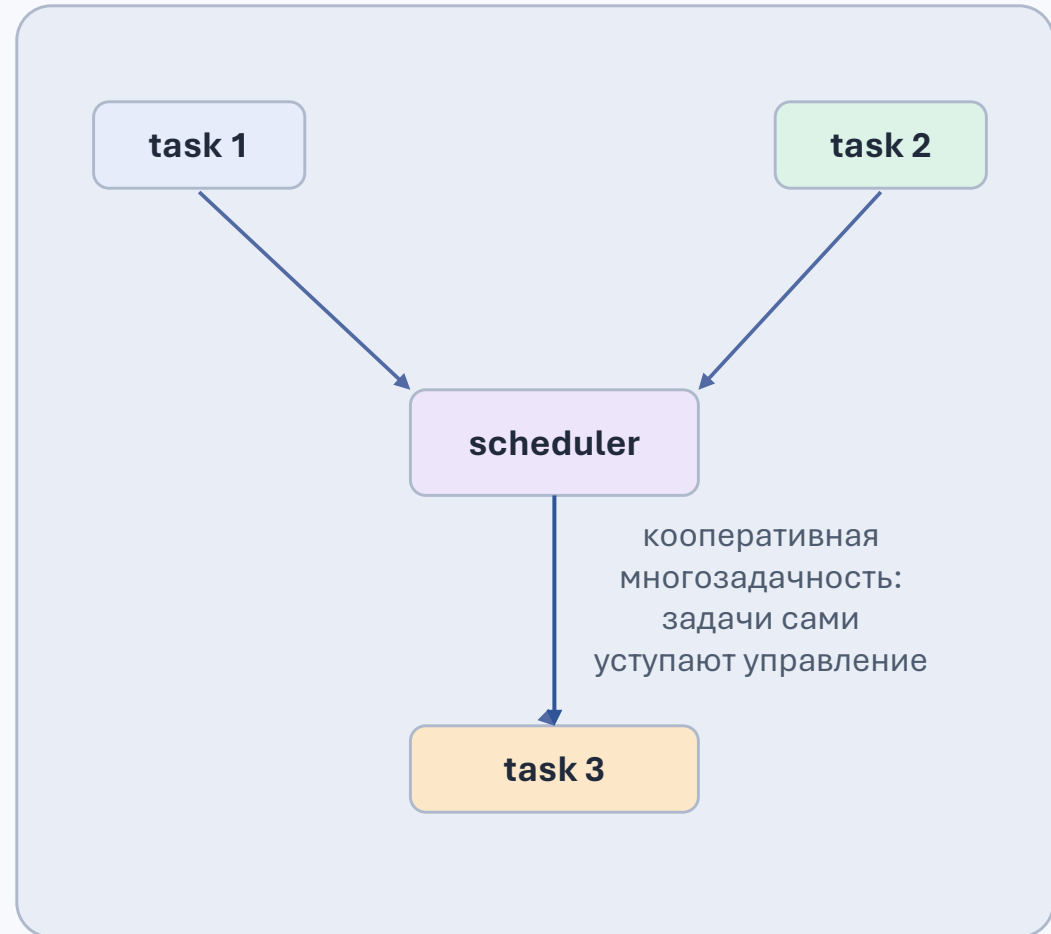
Генераторы в Python исторически использовались как корутины.

Метод `send()` позволяет передавать данные внутрь генератора.



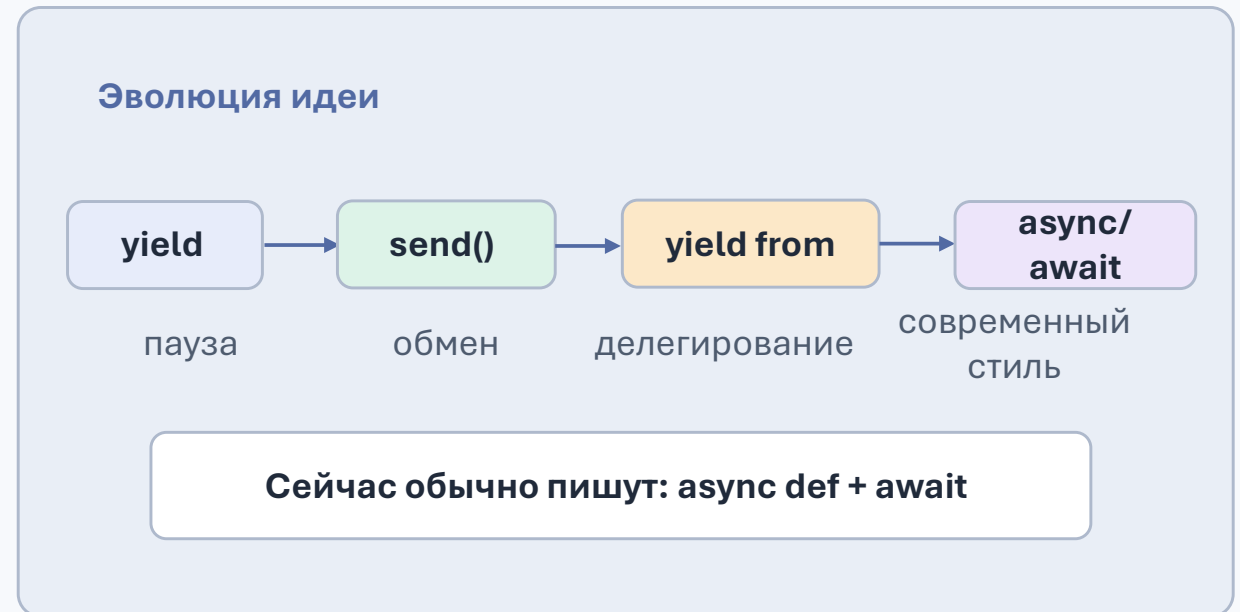
Простейший планировщик задач

Планировщик хранит несколько задач.
Каждый генератор делает один шаг.
После `yield` управление возвращается к планировщику.



От yield from к async/await

Python постепенно ушёл от генераторных корутин к специальному синтаксису. `async/await` делает ожидание явным и понятным. `yield from` был важным промежуточным шагом.

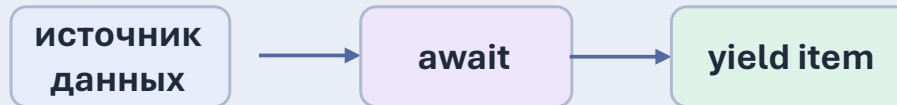


Асинхронные генераторы

Асинхронный генератор объединяет две идеи: `await` и `yield`.

`await` — дождаться операции без блокировки.

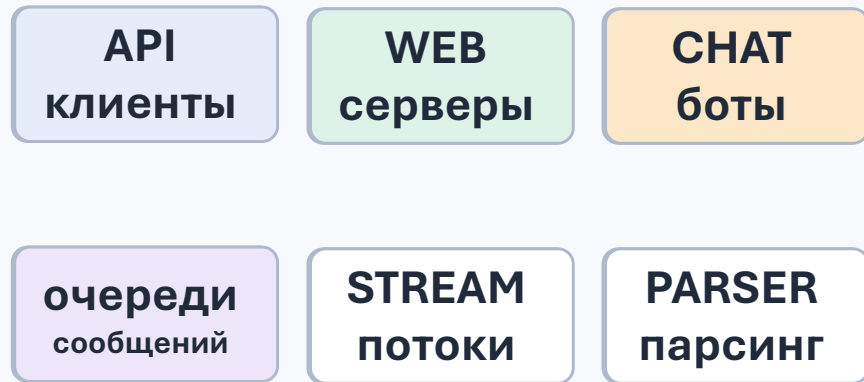
`yield` — отдать очередное значение.



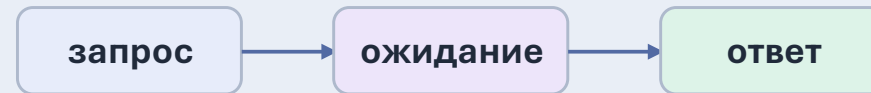
`async for` получает элементы
по мере их появления

пример: сообщения чата, строки из API, поток
событий

Где это применяется



Общий сценарий



Асинхронность особенно полезна, когда много таких операций одновременно.

меньше простоя → выше
отзывчивость

Вывод

Генераторы помогают понять принцип асинхронности.

Ключевое свойство — сохранение состояния между паузами.

Современный Python использует `async/await`, но идея `yield` остаётся важной.

Как это связано

генератор: `yield`



корутина: пауза + продолжение



`async/await`: современный синтаксис

`yield` помогает понять `await`